

交付测试文档

UPFS 交付文档

一、项目功能总览

UPFS 共实现 6 大子系统、35+ 条 shell 命令、20 个系统调用、9 个 JSON API，总代码量约 15000 行 C + 1000 行内嵌前端。

1.1 文件系统层（12 项核心功能）

#	功能点	实现方式
1	虚拟磁盘	546 块 × 512B 镜像文件，mmap 持久化
2	ext2 风格块组	8 组 × 64 块，成组链接法空闲管理
3	XFS 风格动态 Inode	Inode Chunk + Loc/Chunk B+ 树，按需分配，支持叶节点分裂
4	Extent 区段映射	内联 Extent + B+ 树，相邻区段自动合并
5	块缓存	LRU + 哈希双重组织，64 槽，延迟写
6	元数据日志	事务提交 + 挂载回放，32 块日志区
7	VFS 抽象层	操作表分发，支持可插拔文件系统
8	两层打开文件表	用户表(20) → 系统表(40) → Inode 缓存
9	路径解析 (namei)	支持绝对/相对路径、. / .. / ~
10	目录操作	定长目录项(16B)，自动维护 . / ..
11	文件操作	create/open/read/write/close/delete/copy/link
12	UNIX 权限模型	owner/group/other × rwx，root 免检

1.2 内核层（7 项核心功能）

#	功能点	实现方式
13	物理内存管理	128MB 字节数组，位图式页面分配器(4KB 页)
14	32-bit RISC 虚拟机	21 条指令，取指-译码-执行循环
15	进程管理	PCB、进程表(64)、fork/exec/wait/exit
16	时间片轮转调度	100 条指令/时间片，就绪队列
17	系统调用接口	20 个系统调用(文件/进程/环境变量)

#	功能点	实现方式
18	管道通信	匿名管道 + 命名管道(mkfifo), 最多 8 级
19	进程间通信	共享内存、信号机制

1.3 用户管理层（4 项核心功能）

#	功能点	实现方式
20	多用户账户	uid/gid 分配, /etc/passwd 存储
21	口令安全	随机盐值 + 万次迭代哈希, 不可逆
22	用户切换	login/logout/su, 会话隔离
23	环境变量	系统级(/etc/environment) + 用户级(~/.env), 文件持久化

1.4 开发工具链（3 项）

#	功能点	实现方式
24	两遍汇编器	.s 源码 → .upx 二进制
25	C 编译器	词法 → 语法 → AST → 寄存器分配 → 代码生成 → 汇编
26	内置文本编辑器	vim 风格, 支持 :wq/:q 等

1.5 网络服务层（4 项）

#	功能点	实现方式
27	HTTP 服务器	端口 8080, 内嵌 Web 页面
28	WebSocket 终端	/ws/N 多标签终端, fork 子进程
29	WebSocket JSON API	/api 端点, 9 个命令
30	原始 TCP 终端	端口 4096, nc/telnet 接入

1.6 Web 管理界面（3 项）

#	功能点	实现方式
31	文件浏览器	树形目录, 右键菜单, 增删查
32	交互终端	多标签, ANSI 24-bit 颜色, 自动滚动
33	监控仪表盘	块组图、进程表、内存热力图, 3秒刷新

1.7 Shell 命令汇总（35 条）

类别	命令
系统	<code>format</code> <code>mount</code> <code>umount</code>
目录	<code>mkdir</code> <code>cd</code> <code>pwd</code> <code>ls</code>
文件	<code>create</code> <code>write</code> <code>cat</code> <code>rm</code> <code>cp</code> <code>ln</code> <code>stat</code> <code>chmod</code>
用户	<code>useradd</code> <code>login</code> <code>logout</code> <code>su</code> <code>whoami</code> <code>passwd</code> <code>users</code>
进程	<code>asm</code> <code>cc</code> <code>run</code> <code>ps</code> <code>kill</code> <code>mkfifo</code> 命令\ 管道
环境	<code>env</code> <code>export</code> <code>unset</code>
编辑	<code>vim</code>
调试	<code>design_debug</code> (<code>super</code> / <code>inodes</code> / <code>sof</code> / <code>memory</code> / <code>process</code> / <code>all</code>)
其他	<code>help</code> <code>clear</code> <code>exit</code>

二、编译与运行指南

2.1 环境要求

- 操作系统：macOS 或 Linux
- 编译器：GCC / Clang（支持 C17）
- CMake 3.10+
- 无第三方依赖

2.2 编译

```
cd /Users/ac/lab/OS
cmake -B build -S . -DCMAKE_C_FLAGS="-D_GNU_SOURCE"
cmake --build build
```

编译产物： `build/bin/OS_design`

2.3 运行模式

模式一：交互式终端

```
./build/bin/OS_design
```

模式二：Web 管理界面

```
./build/bin/OS_design --serve
# 浏览器打开 http://localhost:8080
# 或 nc localhost 4096 连接原始终端
```

2.4 首次使用流程

1. 启动程序
2. 输入 `format` ← 格式化虚拟磁盘
3. 输入用户名 ← 创建首个用户
4. 输入密码（两次） ← 设置用户口令和 `root` 口令
5. 开始使用 ← 文件系统已挂载，用户已登录

2.5 再次使用流程

1. 启动程序 ← 自动检测到已有磁盘镜像
2. 输入 `mount` ← 挂载文件系统（自动回放日志）
3. 输入用户名密码 ← 登录验证
4. 继续使用 ← 所有数据完好恢复

三、全面测试方案

以下测试用例按功能模块分组，每个用例给出**操作步骤**和**预期结果**。建议在交互式终端模式下逐一执行。

测试 0：编译与启动

```
# 编译（预期：零错误零警告）
cmake -B build -S . -DCMAKE_C_FLAGS="-D_GNU_SOURCE"
cmake --build build

# 启动（预期：显示 UPFS banner，提示输入命令）
./build/bin/OS_design
```

测试 1：格式化与挂载

```
format
```

- 预期：显示“Format complete”，提示创建用户
- 输入用户名 `test`，密码 `123456`（输入两次）
- 预期：显示“User created and logged in”，提示符变为 `test:~ >`

```
umount  
mount
```

- 预期：提示登录，输入 `test` / `123456` 后成功挂载，数据完好

测试 2：目录操作

```
mkdir /mydir  
mkdir /mydir/sub1  
mkdir /mydir/sub2  
ls /
```

- 预期：`ls /` 输出中包含 `mydir` 目录

```
cd /mydir  
pwd
```

- 预期：输出 `/mydir`

```
cd sub1  
pwd
```

- 预期：输出 `/mydir/sub1`

```
cd ..  
pwd
```

- 预期：输出 `/mydir`

```
cd ~  
pwd
```

- 预期：输出 `/home/test`（用户家目录）

```
ls /mydir
```

- 预期：显示 `.` `..` `sub1` `sub2` 四项

测试 3：文件基本操作

```
create /mydir/hello.txt  
write /mydir/hello.txt
```

- 进入编辑器后输入 `Hello World!`，然后 `:wq` 保存退出

```
cat /mydir/hello.txt
```

- 预期：输出 `Hello World!`

```
stat /mydir/hello.txt
```

- 预期：显示 inode 号、类型 regular file、权限 000644、大小、uid/gid

测试 4：文件复制与硬链接

```
cp /mydir/hello.txt /mydir/copy.txt  
cat /mydir/copy.txt
```

- 预期：内容与 `hello.txt` 相同

```
ln /mydir/hello.txt /mydir/link.txt  
stat /mydir/hello.txt
```

- 预期：Links 字段从 1 变为 2

```
stat /mydir/link.txt
```

- 预期：inode 号与 `hello.txt` 相同

测试 5：文件删除与 Inode 回收

```
rm /mydir/copy.txt  
ls /mydir
```

- 预期：copy.txt 消失

```
rm /mydir/link.txt  
stat /mydir/hello.txt
```

- 预期：Links 回到 1（因为 link.txt 被删除）

```
rm /mydir/hello.txt  
create /mydir/new.txt  
stat /mydir/new.txt
```

- 预期：new.txt 的 inode 号与之前 hello.txt 的相同（Inode 回收重用）

测试 6：权限控制

```
chmod /mydir/new.txt 000  
cat /mydir/new.txt
```

- 如果不是 root 用户，预期：权限拒绝

```
chmod /mydir/new.txt 644  
cat /mydir/new.txt
```

- 预期：可以正常读取

测试 7：多用户管理

```
useradd alice 654321  
users
```

- 预期：显示 test 和 alice 两个用户

```
login alice 654321
whoami
pwd
```

- 预期: whoami 输出 `alice`, pwd 输出 `/home/alice`

```
logout
whoami
```

- 预期: 回到之前的用户

```
passwd alice newpass
login alice newpass
```

- 预期: 用新密码登录成功

```
su
```

- 预期: 输入 root 密码后切换到 root

测试 8: 环境变量

```
export MYVAR=hello
env
```

- 预期: env 输出中包含 `MYVAR=hello`

```
unset MYVAR
env
```

- 预期: MYVAR 消失

```
export PERSIST=yes
umount
mount
```


- 登录后执行 `env`
- 预期: `PERSIST=yes` 仍然存在 (环境变量持久化)

测试 9: 汇编程序编译与运行

```
cat /src/hello.s
```

- 预期: 显示预置的汇编源码

```
asm /src/hello.s /bin/myhello  
run /bin/myhello
```

- 预期: 汇编成功, 运行后输出 `hello` 信息

```
ps
```

- 预期: 显示进程列表 (至少有 `init` 进程)

测试 10: C 编译器

```
create /src/test.c  
write /src/test.c
```

- 输入简单 C 代码 (如 `int main() { return 42; }`), `:wq` 保存

```
cc /src/test.c /bin/test  
run /bin/test
```

- 预期: 编译并执行成功

测试 11: 管道

```
ls / | cat
```

- 预期: `ls` 的输出通过管道传递给 `cat` 显示

测试 12：调试命令

```
design_debug super
```

- 预期：显示超级块详情（魔数、Inode 统计、块组状态、动态 Inode 映射 B+ 树信息）

```
design_debug inodes
```

- 预期：显示内存 Inode 缓存池使用情况

```
design_debug sof
```

- 预期：显示系统打开文件表

```
design_debug memory
```

- 预期：显示物理页框分配位图

```
design_debug process
```

- 预期：显示进程控制块信息

```
design_debug all
```

- 预期：依次输出以上所有调试信息

测试 13：数据持久化（关键）

```
# 创建测试数据
mkdir /persist_test
create /persist_test/data.txt
write /persist_test/data.txt PersistenceWorks

# 正常退出
```

```
umount
exit
```

重新启动程序：

```
./build/bin/OS_design
mount
# 登录后
cat /persist_test/data.txt
```

- 预期：输出 `PersistenceWorks`，证明数据完整持久化

测试 14：Web 管理界面

```
./build/bin/OS_design --serve
```

打开浏览器访问 `http://localhost:8080`：

14.1 终端功能

- 预期：中央区域显示终端，可看到 UPFS banner
- 在底部输入框输入 `format`，回车
- 创建用户后，输入 `ls /`
- 预期：正常显示目录列表，ANSI 颜色正确渲染
- 点击 `+` 按钮新建标签页
- 预期：出现 Terminal 2 标签，可独立操作

14.2 文件浏览器

- 预期：左侧面板显示目录树
- 点击目录名展开子目录
- 右键文件 → 选择“View content”
- 预期：弹窗显示文件内容
- 右键目录 → “New file here...” → 输入文件名
- 预期：文件创建成功，树刷新

14.3 监控仪表盘

- 预期：右侧面板显示：
- FILESYSTEM 卡片：Inode 使用数 / 总数
- BLOCK GROUPS 卡片：8 个块组的使用率柱状图
- PROCESSES 卡片：进程列表（PID/Name/State）
- MEMORY 卡片：页面使用统计 + 热力图
- 等待 3 秒后自动刷新

14.4 原始 TCP 终端

```
nc localhost 4096
```

- 预期：显示 UPFS banner，可以执行命令

测试 15：压力测试

```
# 批量创建文件（测试动态 Inode 分配 + B+ 树分裂）
mkdir /stress
create /stress/f01
create /stress/f02
... （创建 60+ 个文件）
create /stress/f60
create /stress/f61

ls /stress
design_debug super
```

- 预期：所有文件成功创建（超过单个 Loc 叶块 63 条限制后，B+ 树自动分裂）
- `design_debug super` 中 Inode 总数应超过 63，Loc 树 level 可能 > 0

```
# 批量删除后再创建（测试 Inode 回收）
rm /stress/f01
rm /stress/f02
rm /stress/f03
create /stress/new1
create /stress/new2
stat /stress/new1
```

- 预期：新文件复用了之前释放的 Inode 号

四、快速验收检查清单

#	检查项	通过标准
1	编译	<code>cmake --build build</code> 零错误
2	格式化	<code>format</code> 成功，自动创建系统目录
3	目录 CRUD	<code>mkdir/cd/pwd/ls</code> 正常
4	文件 CRUD	<code>create/write/cat/rm</code> 正常
5	复制与链接	<code>cp</code> 内容一致， <code>ln</code> 共享 inode

#	检查项	通过标准
6	权限控制	chmod 后权限生效
7	多用户	useradd/login/logout/su 正常
8	持久化	umount → 重启 → mount 数据完好
9	环境变量	export/unset/env 正常，重启后保留
10	汇编执行	asm + run 可编译运行 .s 程序
11	C 编译	cc + run 可编译运行简单 C 程序
12	进程管理	ps 显示进程，run 可执行程序
13	管道	cmd1 \ cmd2 正常传递
14	调试命令	design_debug all 输出完整
15	Web 界面	-serve 后浏览器三栏布局正常
16	Web 终端	浏览器终端可执行命令
17	Web 文件浏览	树形展开，右键菜单可操作
18	Web 监控	仪表盘数据正确，自动刷新
19	动态 Inode	创建 60+ 文件不报错
20	Inode 回收	删除后创建复用旧 inode 号